MEMORIA PROYECTO FINAL

FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL

ADRIANA LORENZO MINGORANCE

ÍNDICE

1. Introducción 2
2. Diseño general del sistema 2
3. Parte 1: Agente inteligente en entorno parcialmente observable 4
   3.1. Representación del entorno (map.py) 4
   3.2. Base de conocimiento y razonamiento lógico (knowledgeBase.py) 5
   3.3. Modo A: control manual guiado por lógica 6
   3.4. Modo B: exploración automática por algoritmos de búsqueda: BFS 8
4. PARTE 2.1. Agente inteligente en entorno parcialmente observable 10
   4.1. Inferencia bayesiana 10
   4.2. Representación del entorno (palacio_environment.py) 11
   4.3. Modo A: control manual con razonamiento bayesiano 12
   4.4. Modo B: planificación automática mediante búsqueda informada A* 13
5. PARTE 2. 2. Cruzar el río 16
   5.1. Descripción del problema 16
   5.2. Implementación del entorno (class RiverEnvironment) 16
   5.3. Modelado del MDP (class RiverMDP) 17
   5.4. Algoritmo de Value Iteration 18
   5.5. Simulación siguiendo la política óptima 19
6. CONCLUSIÓN 21
7. BIBLIOGRAFÍA 21

1. Introducción

En este proyecto, buscamos dar solución al problema planteado: 'Buscando al coronel Kurtz'. En él, ayudamos a nuestro Capitán Willard a encontrar al coronel Kurtz en un palacio lleno de trampas, basándonos en las percepciones del ambiente y ayudándonos en los elementos estudiados en nuestra asignatura de Fundamentos de Inteligencia Artificial.

Para ello, se implementa un agente inteligente que opera en un entorno parcialmente observable, cuyo objetivo es explorar el entorno, rescatar a Kurtz y alcanzar la salida, evitando peligros como precipicios y un soldado enemigo, o bien, en la segunda parte, otra serie de trampas.

Por ello, la parte 1 del proyecto se centra en resolver el problema de dos maneras bien diferenciadas, y las cuales hemos aprendido en la presente asignatura: primero, un modo manual, centrado en el razonamiento lógico, el cual, mediante percepciones, nos ayuda a descartar peligros; y en segundo lugar, un modo automático basado en los algoritmos de búsqueda estudiados en clase (concretamente, en el BFS).

Por otro lado, la parte 2, sigue al principio una estructura simular, tratando de resolver el problema igualmente mediante dos maneras: el modo manual, y el modo automático (basándonos en este caso, el algoritmo de búsqueda informada A* con heurística). No obstante, en esta segunda parte se presenta no solo una modificación de las posibles trampas (ahora fuego, pinchos o dardos), sino también de las percepciones de estas y de la forma de abordarlas, ya que, en este caso, planteamos mapas de probabilidad que siguen las fórmulas bayesianas estudiadas en clase, para poder guiar al agente.

Por último, resolvemos la parte 2 'cruzar el río', donde nuestro Capitán, después de rescatar a Kurtz, deberán intentar cruzar un río que cuenta con posibles peligros. Para ello, hemos modelado un modelo de Markov (MDP) tal y como estudiamos en la última parte de la asignatura.

2. Diseño general del sistema

Para poder resolver los problemas correctamente, distinguimos una serie de ficheros asociados a cada parte.

En la parte 1, nos hemos apoyado en 4 ficheros '.py' bien diferenciados. Cada uno, mediante sus clases, métodos y funciones, realiza una tarea específica que permiten la correcta resolución del juego en cada iteración y según el modo de juego escogido.

Los ficheros de la parte 1 son así: map.py, knowledgebase.py, search.py, kurtz.py

En knwoledgeBase.py, hemos implementado (mediante la clase KnowledgeBase), toda aquella información proporcionada en el enunciado; desde cada elemento y personaje hasta las acciones y reacciones posibles que puede tomar el agente cuando se topa con alguno de ellos. Así, almacena no sólo información cierta y posible sobre el entorno, sino

que también aplica reglas de inferencia lógica tratadas en clase, que nos ayudarán en el futuro para deducir nueva información y así resolver el problema.

Con map.py, hemos podido realizar la representación del entorno real: generando así un mapa aleatorio en cada iteración tal y como estipulaba el enunciado; tratando además el estado del agente, los perceptos, y la ejecución de las acciones; lo cual nos ayudará también principalmente a la ejecución del juego en sí, en ambos modos de resolución.

De igual manera, hemos creado search.py: un fichero auxiliar para profundizar y aislar más definidamente el modo B de resolución (mediante algoritmos de búsqueda). En él, incluimos algoritmos de búsqueda utilizados para la planificación de rutas seguras, concretamente BFS sobre celdas conocidas como seguras.

Por último, tenemos kurtz.py como nuestra columna vertical del proyecto, el cual actúa como controlador principal del sistema: permitiendo ejecutar el agente en modo manual o automático y coordinado la interacción entre el entorno, la base de conocimiento y la búsqueda.

Por otro lado, en la parte 2, al partir de un problema similar, nos hemos apoyado en todo momento en la estructura y resolución de la parte 1, por ello, siguiendo su modelo, hemos resuelto el problema con los ficheros: palacio_environment.py, palacio_bayes.py, palacio_search.py, palacio.py.

En primer lugar, palacio_bayes.py implementa el razonamiento probabilístico del agente mediante inferencia bayesiana, manteniendo y actualizando distribuciones de probabilidad sobre nuevas trampas, soldado, Kurtz y salida a partir de las percepciones recibidas en cada turno.

Asimismo, palacio_environment.py define el entorno del problema como una cuadrícula parcialmente observable, gestionando la generación del mapa, las percepciones del agente, las acciones disponibles y la dinámica del juego.

Después, el palacio_search.py, contiene los algoritmos de planificación y apoyo a la toma de decisiones automáticas, implementando búsqueda informada A* sobre el tablero, utilizando como coste el riesgo estimado a partir de mapas bayesianos.

Y de igual manera, palacio.py actúa como script principal que integra entorno, inferencia y planificación, coordinando el ciclo percepción-razonamiento-acción tanto en modo manual como en automático

Por último, para el problema de cruzar el río, hemos compactado todo en un único fichero: river_mdp.py para facilitar la comprensión y su independencia del resto de ficheros. En él, planteamos tanto el entorno y tablero de juego, como la definición del MDP con sus respectivas acciones, estados, transiciones, etc que veremos más adelante.

3. Parte 1: Agente inteligente en entorno parcialmente observable

En esta primera parte del proyecto se ha desarrollado un agente capaz de razonar bajo incertidumbre y de planificar movimientos seguros en un entorno parcialmente observable. La separación entre razonamiento lógico y planificación permite comparar un control manual con un comportamiento autónomo basado en búsqueda, estableciendo un gran afianzamiento del temario estudiado en clase desde un punto de vista más programático y real, así como una base sólida para la parte 2.

3.1. Representación del entorno (map.py)

El entorno se modela como una cuadrícula de tamaño nxn, donde cada celda puede contener distintos elementos, como precipicios, el soldado, Kurtz o la salida. El agente comienza siempre en la posición (0,0), desde la cual debe explorar el entorno de manera progresiva.

En este fichero, hemos definido en primer lugar las acciones que puede realizar el capitán, las cuales son las cuatro direcciones cardinales (con WASD), así como lanzar una granada en una de esas direcciones (en este caso, con IJKL), o bien salir (E).

Después, apoyándonos en la clase Environment hemos definido todos los elementos, así como el método generate_random_map: encargado de colocar de manera aleatoria los elementos en el mapa en cada iteración, garantizando que cada uno ocupe una única celda válida.

De igual manera, se implementan dos métodos de visualización. El método plantilla_truth permite mostrar el mapa real completo (que no se usa en el modo de juego, solo para ver al final de la partida donde estaba cada elemento) , mientras que plantilla_conocida, representa el mapa parcial que el agente ha ido construyendo a partir de su base de conocimiento durante la ejecución del juego. Esta es la que se va mostrando según vamos jugando e infiriendo sobre la información de las celdas.

Los métodos _move, lanzar_granada y get_adyacentes, nos ayudan auxiliarmente a movernos por el tablero, lanzar al granada en cada dirección y establecer las consecuencias de matar al soldado, etc; y obtener las celdas adyacentes a nuestra nueva posición.

Con get_percepciones, el agente recibe un conjunto de perceptos booleanos: brisa, ronquido, resplandor, pared superior, inferior, izquierda, derecha y grito; ya definidos en el enunciado. En este método los definimos y estipulamos cuándo deben aparecer como True en la partida.

Por último, el método step, actúa como interfaz principal entre el agente y el entorno. Con él, se devuelve un diccionario ‘info’ que nos permite saber cuándo hemos muerto/ganado, estipulando cada caso posible de muerte o saber si hemos usado la granada, así como la consecuencia de usarla (el grito, matar al soldado etc), indicando así posibles estados terminales.

3.2. Base de conocimiento y razonamiento lógico (knowledgeBase.py)

Apoyándonos en la clase KnowledgeBase definida dentro de este fichero, vemos que la base de conocimiento mantiene una separación clara entre conocimiento cierto (celdas visitadas, seguras y peligros confirmados) y conocimiento posible (celdas que podrían contener peligros). Esta distinción permite razonar bajo incertidumbre de forma progresiva.

El conocimiento se estructura así en dos niveles complementarios:

1. Conjuntos globales de posibilidad, que indican qué celdas podrían contener un peligro: posible_precipicios, posible_soldier.

Estos conjuntos permiten, mediante una agruapcion set(), identificar rápidamente qué celdas no pueden considerarse seguras en un momento dado.

2. Restricciones lógicas asociadas a perceptos concretos, que se almacenan mediante diccionarios: brisa_constraints, sold_constraints.

Cada entrada de estos diccionarios asocia una celda en la que se ha percibido una brisa o un ronquido con el conjunto de celdas adyacentes que podrían ser la causa de dicho percepto. De este modo, cada percepto se traduce en una restricción lógica independiente del tipo “al menos una de estas celdas contiene el peligro correspondiente”.

Esta doble representación es necesaria porque los conjuntos globales de posibilidad indican qué celdas podrían ser peligrosas, pero no explican por qué ni desde qué percepto se ha inferido dicha posibilidad. Para poder realizar deducciones lógicas más profundas, es imprescindible conservar la relación entre perceptos y sus causas potenciales.

Por ello, la actualización de la base de conocimiento se realiza mediante el método update_with_percept, que constituye el núcleo del razonamiento lógico del agente. En cada turno, este método recibe la posición actual del agente y el conjunto de percepciones observadas, y actualiza el conocimiento disponible en función de reglas deterministas.

En primer lugar, la celda actual se marca como visitada y segura (apoyándonos en métodos auxiliares como mark_visited y get_celdas_adyacentes), ya que el agente ha sobrevivió al entrar en ella. A continuación, se analizan las celdas adyacentes en función de cada percepción y según lo marcado en el enunciado. Por ejemplo, si no se percibe brisa, se puede deducir que ninguna de las celdas adyacentes contiene un precipicio, por lo que estas se marcan como seguras. De forma análoga, la ausencia de ronquido permite descartar la presencia del soldado en las celdas vecinas.

En caso contrario, cuando se percibe brisa o ronquido, no se puede identificar directamente qué celda lleva el peligro. En estos casos, añadimos restricciones lógicas que asocian la percepción observada con el conjunto de celdas adyacentes que podría ser la causa. Estas restricciones se almacenan en los diccionarios brisa_constrains y sold_constrains (definidos previamente en el init de la clase), permitiendo así conservar relación entre percepciones y posibles causas.

A partir de estas restricciones, el agente aplica inferencia por descarte mediante los métodos infer_precipicios e infer_soldado. En particular, si una restricción asociada a una percepción (por ejemplo, una brisa) contiene inicialmente varias celdas candidatas, estas

se van eliminando a medida que se obtiene nueva información (celdas visitadas, celdas marcadas como seguras o peligros descartados por otras percepciones). Así, si una restricción queda reducida a una única celda candidata, el peligro correspondiente se confirma en dicha celda y se descarta en el resto. Este proceso permite transformar conocimiento posible en conocimiento cierto de forma incremental.

Sucede de igual manera con el soldado, ya que una vez que se infiere con certeza la posición del soldado enemigo, se actualizan todas las restricciones asociadas a los perceptos de ronquido (se vuelve False ya para el fin de la partida), eliminando cualquier celda incompatible y manteniendo únicamente aquellas que son coherentes con la posición confirmada del soldado.

Con este mecanismo de inferencia, nos basamos exclusivamente en reglas lógicas deterministas, y no implica planificación ni búsqueda. La KB así se limita a razonar sobre la información disponible, separando claramente el proceso de inferencia del proceso de toma de decisiones, que abordaremos bien de modo manual o automático.

3.3. Modo A: control manual guiado por lógica

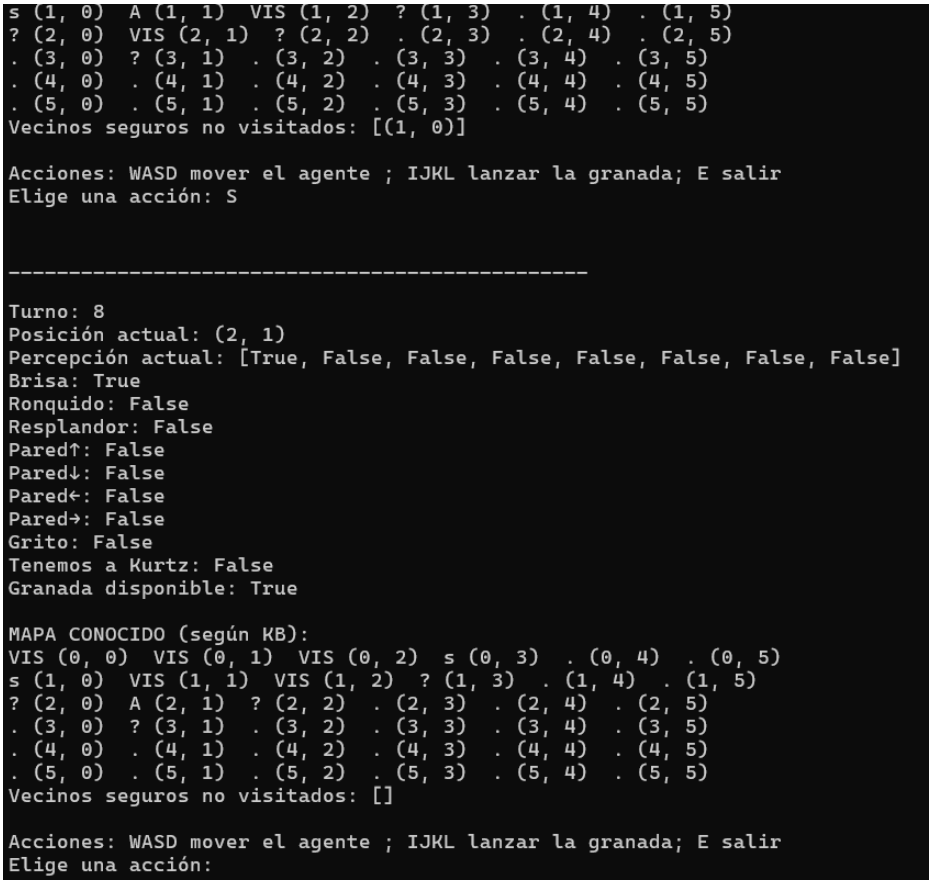
En el modo manual, el usuario controla directamente las acciones del agente mediante el teclado. Para ello, hemos provisto las teclas WASD (como aconsejaba el enunciado) para desplazar el agente, y las teclas IJKL para dirigir la dirección en la que lanzar la granada si decidimos hacerlo. Sin embargo, el proceso de razonamiento lógico y la actualización de la base de conocimiento se realizan de forma automática tras cada percepción.

Para ello, en el script principal kurtz.py, nos basamos en la función run_manual, la cual se encarga de que en cada iteración se sigan los mismos pasos. Primero, se percibe el entorno que mostramos al inicio con una lista de booleanos, (que indica True si estamos percibiendo ese elemento en concreto y False si no):

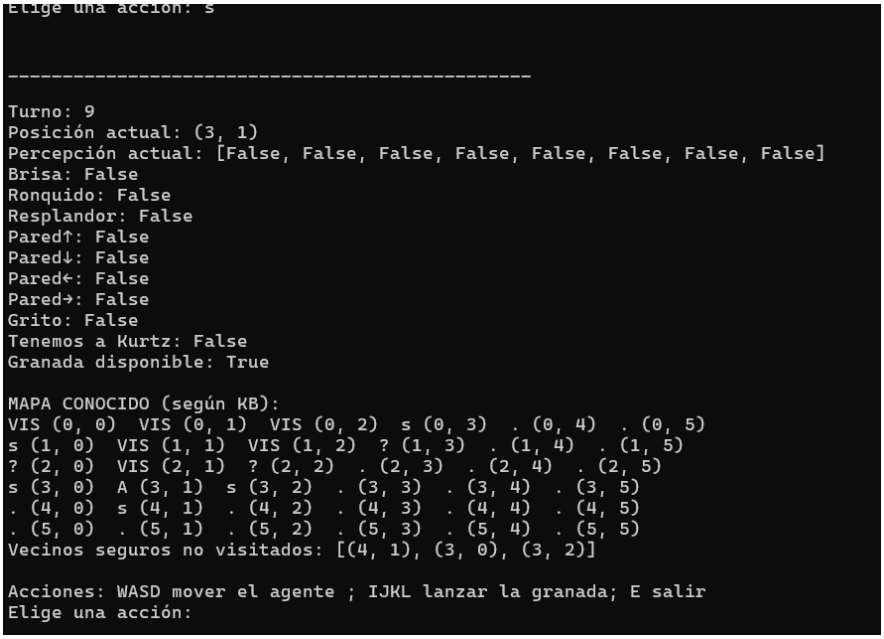
percept_nombres = ['Brisa', 'Ronquido', 'Resplandor', 'Pared↑', 'Pared↓', 'Pared←', 'Pared→', 'Grito'].

Después, razonamos sobre estas percepciones llamando al método update_with_percept de la base de conocimiento que hemos mencionado anteriormente. De igual manera, mostramos el mapa que va conociendo el agente según la información que recoge e infiere de cada iteración. De igual manera, añadimos un método opcional que permite sugerirle al agente ciertos vecinos donde poder moverse que sabemos con certeza que son celdas seguras. Por último, le pedimos al usuario la acción que desea realizar y actuamos en base a esta y a la celda a la que nos lleva tal acción, mostrando, cuando se termina la jugada, el mapa real del palacio desde el que partíamos.

Un ejemplo de una iteración sería en este caso:



Vemos como al pulsar la tecla ‘S’ nos hemos movido hacia abajo, donde se ha detectado cierta brisa (percepción Brisa: True) y nuestras casillas adyacentes se marcan como posibles peligros, alertándonos de que no deberíamos ir por ahí porque cualquiera puede ser un posible precipicio:



No obstante, al ‘arriesgarnos’ e ir por ejemplo hacia abajo, vemos que no había precipicio. Con ello, la base de conocimiento actualiza automáticamente que ya no hay brisa y marca esa celda como segura. Aún así, sigue manteniendo el posible peligro sobre las que teníamos antes aunque ay no percibamos la brisa, mostrando así, como almacena la información e infiere sobre el entorno.

Este modo permite observar de manera transparente cómo el agente interpreta los perceptos, actualiza su conocimiento y deduce nueva información sobre el entorno. Resulta especialmente útil para validar el correcto funcionamiento de la base de conocimiento y para comprender el impacto de cada acción en el estado interno del agente.

3.4. Modo B: exploración automática por algoritmos de búsqueda: BFS

Para terminar de implementar este modo, nos hemos ayudado de un script adicional: search.py, donde hemos definido funciones encargadas exclusivamente del cálculo de rutas y de la traducción de dichas rutas a acciones ejecutables en el entorno.

En el modo automático, el agente explora el entorno de forma autónoma mediante un proceso de planificación basado en el algoritmo de búsqueda BFS estudiado en clase. La búsqueda se restringe exclusivamente a las celdas que la KB considera seguras, evitando así cualquier movimiento que implique riesgo.

La función bfs_path implementa un algoritmo de BFS clásico para calcular un camino seguro. Para ello, hemos seguido la dinámica FIFO característica del algoritmo, definiendo así una cola queue y un diccionario parent, que permite reconstruir el camino una vez alcanzado el objetivo. En cada expansión, se consideran sólo vecinos transitables aquellas celdas que la KB identifica como seguras mediante el método get_safe_neighbors. Si se encuentra un camino seguro, la función devuelve una lista ordenada de celdas desde la posición inicial hasta el objetivo. En caso contrario, devuelve None, indicando que no existe una ruta segura conocida.

Una vez calculado el camino, la función cell_to_action, se encarga de ‘traducir’ el desplazamiento entre celdas consecutivas en una acción válida del entorno (up, down, left o right).

La función manhattan calcula la distancia de manhattan entre dos celdas de la cuadrícula. Dado que el agente solo puede desplazarse en direcciones cardinales, esta medida resulta adecuada para estimar la cercanía entre posiciones.

Volviendo así al script principal kurtz.py, definimos por un lado una función auxiliar para saber hacer la selección del objetivo de exploración: choose_goal_safe_unvisited.

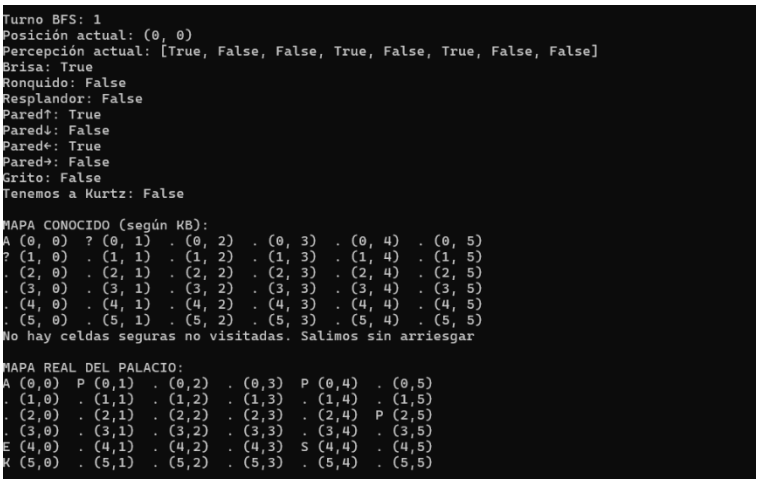
Primero, se priorizan los vecinos seguros no visitados inmediatos a la posición actual del agente. Si no existen vecinos seguros no visitados, el método busca de forma global celdas que la base de conocimiento considera seguras pero que aún no han sido visitadas. Entre estas candidatas, se descartan aquellas asociadas a peligros confirmados y se selecciona la más cercana según la distancia Manhattan.

Si no existe ninguna celda segura no visitada, la función devuelve None, y decimos que el agente no puede continuar explorando sin asumir riesgos.

Por último, canalizamos toda esta información en la función principal: run_search_bfs. En cada iteración, el agente obtiene las percepciones del entorno y actualiza su base de conocimiento mediante inferencia lógica (esto es similar al modo manual). A continuación, se selecciona un objetivo seguro no visitado y se planifica un camino hasta él utilizando BFS.

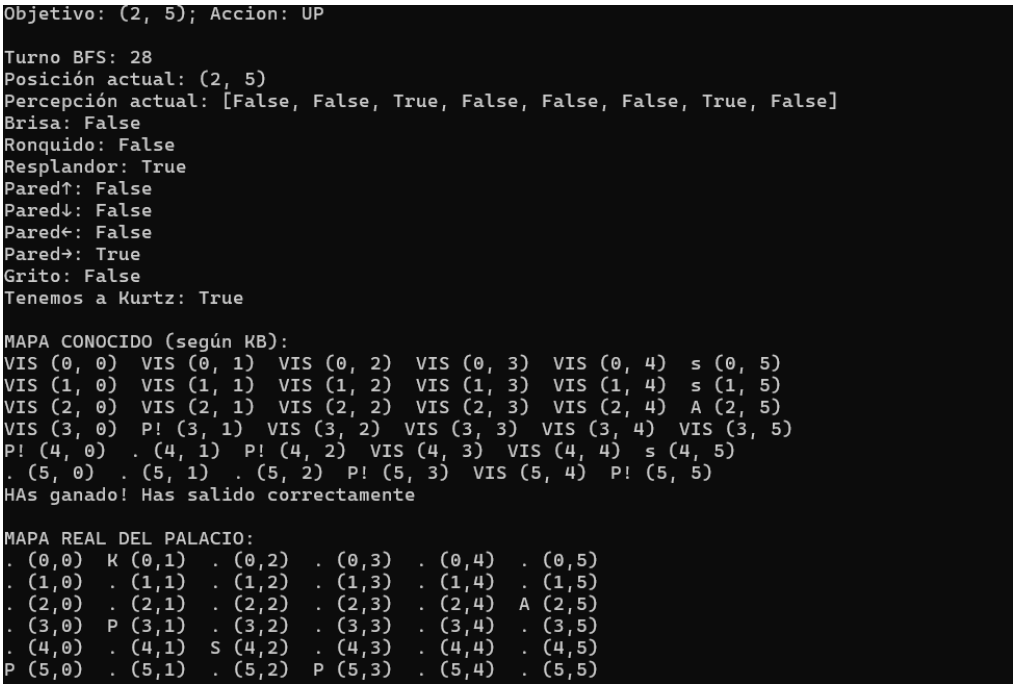
Aunque el BFS calcula un camino completo, el agente ejecuta únicamente el primer paso del plan, ya que tras cada acción el entorno puede cambiar, por lo que el proceso se replantea en cada turno. Si no existen objetivos seguros disponibles, el agente preferirá finalizar la exploración sin arriesgarse.

Vemos un ejemplo:



Vemos como esta rodeado de posibles peligros con la brisa, por lo que prefiere salir.

Otro ejemplo:



Aquí ha encontrado el camino a seguir hasta llegar al objetivo, pasando por K (Kurtz, tras rescatarle).

En conjunto, el modo B combina razonamiento lógico infiriendo con la KB las celdas seguras, con el cálculo de rutas sobre dichas celdas mediante el algoritmo de búsqueda BFS. Con esta implementación, resolvemos satisfactoriamente

4. PARTE 2.1. Agente inteligente en entorno parcialmente observable

En esta segunda parte del proyecto se amplía el modelo desarrollado en la Parte 1, sustituyendo el razonamiento lógico determinista por un razonamiento bayesiano bajo incertidumbre cuantificada. El objetivo sigue siendo el mismo, rescatar al coronel Kurt, aunque ahora con nuevas trampas y entorno, pero siempre buscando tomar decisiones informadas basadas en probabilidades y aceptando riesgos controlados cuando sea necesario.

Con ello, resumimos cada iteración en:

1. Generar los 3 mapas (trampas / enemigo / salida)
2. Se actualizan a posteriori cada vez que te mueves
3. Con eso puedes moverte (modo A: guiado por el usuario o modo B: búsqueda informada)

4.1. Inferencia bayesiana

El fichero palacio_bayes.py implementa el núcleo del razonamiento probabilístico del agente. A diferencia de la KB de la parte 1, este módulo mantiene distribuciones de probabilidad explícitas sobre la posible localización de cada elemento peligroso.

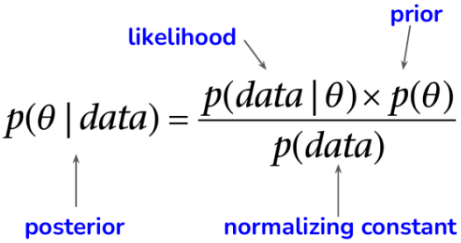
Para cada tipo de elemento del entorno, se mantiene una matriz nxn con los métodos _init_uniform_prior e _init_priors: PF (probabiliad de fuego); PP (probabilidad de pinchos); PD (probabilidad de dardos); PM (probabilidad de militar/enemigo); PS (probabilidad de salida).

Cada matriz representa una distribución de probabilidad normalizada sobre el tablero, reflejando el grado de creencia del agente sobre la presencia de dicho elemento en cada celda.

Para ello, sobre la clase PalacioBayes, tenemos el método principal update, que recibe la posición actual del agente y el vector de percepciones observado en ese turno. A partir de esta información, se actualizan las distribuciones aplicando razonamiento bayesiano.

La lógica general, implementada con el método update_with_evidence es la siguiente:

-Si una percepción no se observa, se reduce (o se pone a 0), la probabilidad del elemento correspondiente en las celdas adyacentes.
-Si una percepción se observa, se refuerza la probabilidad se refuerza la probabilidad de dicho elemento en las celdas adyacentes compatibles con la percepción, aplicando bayes:



Donde la likehood según el enunciado está definida como:

$$p(e_{ij}^{\tau} | \tau_{k\ell}) = \begin{cases} 1 & (k,l) \in adj(i,j) \cup \{(i,j)\}, \\ 0 & \text{otherwise.} \end{cases}$$

Por lo que resumimos el calculo de la posterior de cada celda, con que será igual al prior (al multiplicar prior*1) si está en la zona correspondiente, o será 0.

La celda actual se considera siempre segura, ya que el agente ha sobrevivido al entrar en ella.

Tras cada actualización, llamamos al método normalize, y renormalizamos las matrices para garantizar que representan distribuciones de probabilidad válidas.

Para facilitar la toma de decisiones, se define un método axuliar: trampa_cualquiera que combina las probabilidades de los distintos peligros en una única medida de riesgo: P(trampa) = P(F) + P(P) + P(D)

Con otra serie de métodos similares a los de la parte 1 (make_grid, vecinos_dentro_tablero, adj_y_mismo) así como con los nuevos: get_required_maps y print_required_maps; creamos los 3 mapas de probabilidad solicitados; el de las trampas, el del soldado enemigo y el de la salida. Estos mapas van actualizando su distribución de probabilidad conforme el agente se va moviendo por los mismos, aplicando la inferencia bayesiana estudiada y mencionada.

4.2. Representación del entorno (palacio_environment.py)

El fichero palacio_environment.py define el entorno físico del problema y cumple un papel análogo al fichero map.py de la parte 1.

El entorno se modela nuevamente como una cuadrícula de tamaño nxn, donde cada celda puede contener bien una trampa o varias, bien alguno de estos elementos: M militar, Kurtz o la salida, también posible que alguno de estos 3 últimos se encuentren en la misma celda. El agente comienza siempre en la posición (0,0).

El método _generate_random_map se encarga de colocar todos los elementos de manera aleatoria, asegurándose de que ningune se coloque en la posición inicial del agente. Este

procedimiento asegura variabilidad entre partidas y mantiene el carácter parcialmente observable del entorno.

El método get_percepciones devuelve un vector de percepciones booleanos (al igual que en map.py), que informa al agente sobre la presencia cercana de elementos relevantes: [eF, eP, eD, eM, eS, Pared↑, Pared↓, Pared←, Pared→, Grito]

Las percepciones se definen de forma local, afectando únicamente a la celda actual y sus adyacentes (ayudándonos del método auxiliar get_adyacentes).

El método step actúa como interfaz principal entre el agente y el entorno. A partir de una acción, actualiza el estado del entorno y devuelve un diccionario info que indica si el agente ha muerto, ha ganado la partida, o el resultado de lanzar la granada, en caso de hacerlo.

Junto con los métodos lanzar_granada, _move, plantilla_truth, … nos podemos desplazar de manera simultánea por los 3 mapas generados, así como visualizar nuestros movimientos/recorrido y lanzar la granada en caso de que notemos al soldado cerca. Todo esto funciona de manera similar a la parte 1.

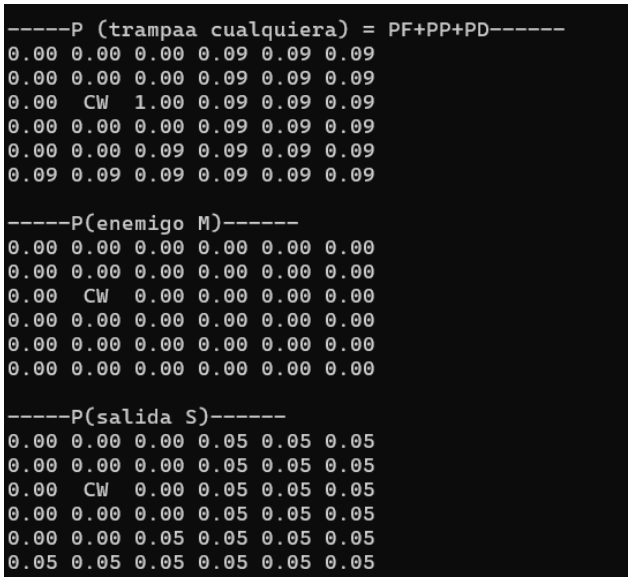
4.3. Modo A: control manual con razonamiento bayesiano

En el modo manual, el usuario controla directamente las acciones del agente mediante teclado (con las teclas WASD e IJKL para lanzar granada), de forma análoga a la parte 1. Sin embargo, el razonamiento interno del agente se basa ahora en inferencia bayesiana y los 3 mapas de probabilidad creados, en lugar de en un solo mapa con lógica determinista.

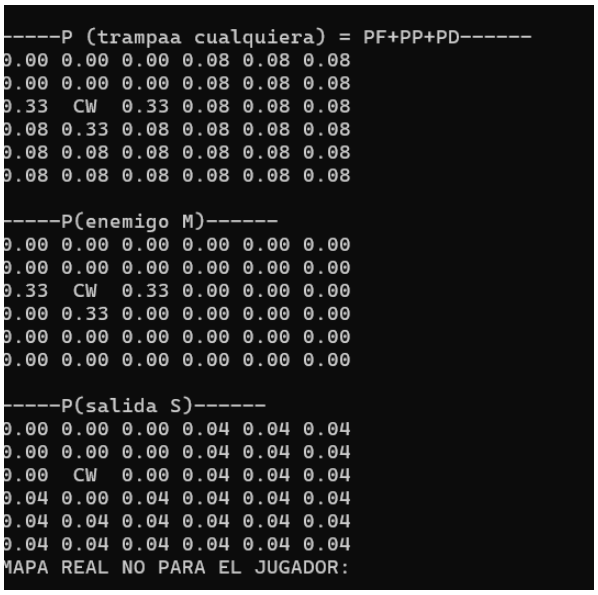
Por ello, en cada turno: percibimos el entorno, se actualizan las distribuciones de probabilidad mediante el módulo bayesiano, mostramos los mapas probabilísticos exigidos y el usuario decide la acción a ejecutar en función de éstos.

Este modo permite observar de forma transparente cómo las probabilidades evolucionan con cada percepción, facilitando la comprensión del razonamiento bajo incertidumbre.

Un ejemplo sería:



Vemos como cada mapa muestra la probabilidad bien de encontrar alguna trampa cualquiera, bien un soldado, o la salida, y de ahí nos vamos guiando para avanzar. Se va actualizando en cada iteración ya que si por ejemplo nos 'arriesgarnos' a ir abajo y a la izquierda y caer en una posible trampa y no ocurre nada:



La celda que antes tenía un 0.33 de la derecha se actualiza y ahora tiene una probabilidad de 1 de estar ahí la posible trampa, que se mantiene para el resto de las iteraciones. Lo mismo con el soldado y la salida.

4.4. Modo B: planificación automática mediante búsqueda informada A*

En el modo automático, el agente actúa de forma autónoma combinando inferencia bayesiana con búsqueda informada A*. Nos apoyaremos nuevamente en un fichero auxiliar: palacio_search,py para planificar las rutas.

A diferencia del BFS de la parte 1, que solo permitía desplazamientos completamente seguros, ahora el agente puede entrar en celdas con riesgo limitado por un umbral p.

Dentro de palacio_search.py tenemos nuevamente la función cell_to_action, que cumple el mismo rol que la parte 1: traducir un desplazamiento entre dos celdas consecutivas en una acción ejecutable por el entorno. También contamos nuevamente con la función manhattan, ya que la heurística empleada en A* es la distancia Manhattan, adecuada ya que el agente solo puede moverse en direcciones cardinales, y nunca sobreestima el coste real (heurística admisible). Esta función implementa así directamente el término h(n) del algoritmo A*.

La función risk_plantilla es clave en la parte 2, ya que conecta inferencia bayesiana con planificación. A partir de las distribuciones probabilísticas mantenidas por PalacioBayes, se construye una nueva matriz: risk(i,j) = P(trampa) + P(soldado) en (i,j). El riesgo se satura en 1 para mantenerlo como probabilidad válida.

Este mapa de riesgo permite evaluar cuantitativamente la peligrosidad de cada celda, lo cual nos facilitará evaluar que la peligrosidad esté por debajo del umbral, así como el cálculo de la ruta a tomar.

Ahora sí, con la función a_path, implementamos el algoritmo A* clásico, con dos adaptaciones fundamentales:

1. Restricción por umbral de riesgo (ayudándonos de una función auxiliar allowed), donde el agente solo expande celdas cuyo riesgo esté por debajo de un umbral p, salvo (opcionalmente) el objetivo.
2. La función de coste informada por riesgo. Definimos así el coste de transición como: g(n') = g(n) + (1 + Alpha*riesgo(n')) , definido el coste de paso como: coste_paso = 1.0 + alpha * risk[vecino[0]][vecino[1]]

El término constante representa el coste del movimiento, y hemos añadido Alpha para penalizar las celdas más peligrosas. Esto fuerza a A* a preferir caminos más largos pero menos arriesgados.

La función de evaluación queda así, tal y como hemos estudiado: **f(n) = g(n) + h(n)**

Por último, con la función de best_neighbor_fallback, cuando A* no encuentra una ruta válida bajo el umbral de riesgo, el agente selecciona el vecino inmediato con menor riesgo, garantizando que el agente no queda bloqueado y se tome la decisión local menos peligrosa posible.

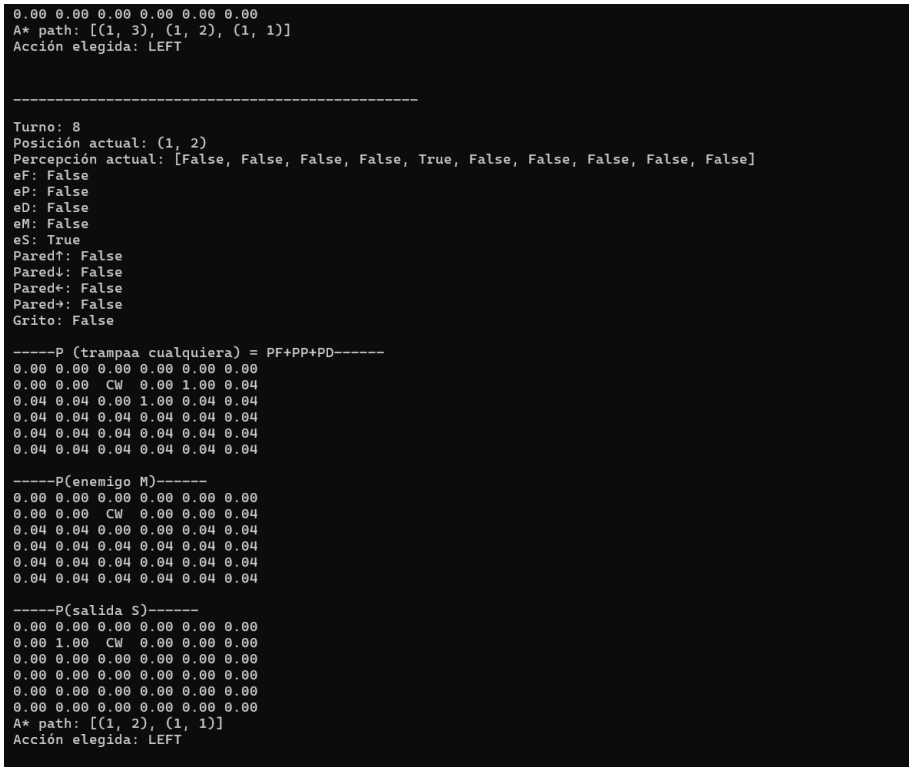
Todo esto queda recogido e implementado con la función run_auto_a de palacio.py, que coordina todo el comportamiento del agente en cada turno, reutilizando la estructura general del modo automático de la parte 1.

El ciclo nuevamente sigue que en cada iteración: el agente percibe el entorno (env.get_percepciones), actualiza sus creencias bayesianas (bayes.update), muestra los mapas probabilísticos requeridos, decide un objetivo, planifica una acción con A*, y ejecuta solo el primer paso del plan, permitiendo así replantear la planificación tras cada nueva evidencia.

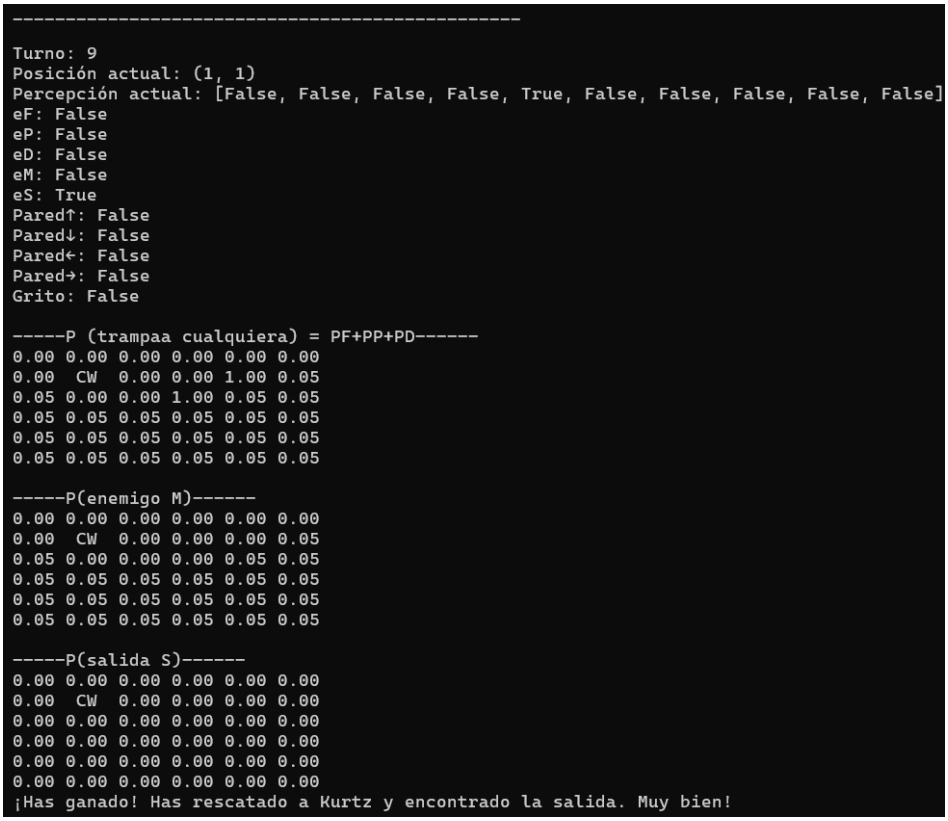
Para al selección del objetivo, ya que no se nos pide definir ningún tipo de probabilidad sobre Kurtz, el agente buscará su celda explorando las no visitadas con riesgo inferior al umbral p, priorizando las más seguras. Una vez rescatado Kurtz, el objetivo pasa a ser la celda con mayor probabilidad de contener la salida (usando así la función auxiliar argmax_cell sobre PS).

Esta lógica garantiza que el agente no intente salir sin Kurtz, respetando las reglas del problema.

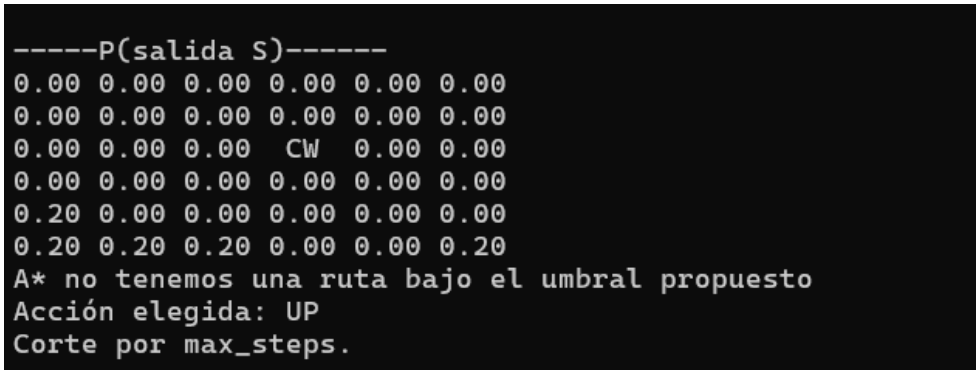
Un ejemplo de este tipo de resolución es:



Muestra como la posición actual (1,3) decide ir hacia el vecino (1,2) ya que lo ha seleccionado como el que menos riesgo tiene y con distancia de mahattan = 1, y así sucesivamente:



También puede suceder que no se encuentre ninguna ruta bajo el umbral propuesto, y con nuestro límite de max_steps prefiramos acabar el juego antes de que entre en un bucle infinito:



Con esta implementación pretendemos resolver nuevamente el rescate a Kurtz empleando el método de los algoritmos de búsqueda visto en clase, el cual, al ser de búsqueda informada, vemos que es bastante más óptimo y rápido.

5. PARTE 2. 2. Cruzar el río

5.1. Descripción del problema

En este apartado, se modela el problema de cruzar un río como un Proceso de Decisión de Markov (MDP).

El objetivo del agente (el capitán y coronel CWCK) es llegar desde la orilla inicial, hasta la casilla de salida E de la casilla final, evitando las islas (obstáculos) y minimizando el número de movimientos. Además, el río presenta un corriente estocástica que puede empujar al agente hacia abajo dependiendo de al columna en la que se encuentre.

Hemos modelado así el problema como un Proceso de Decisión de Markov, implementando tanto el entorno como el algoritmo de Value Iteration estudiando, así como una simulación final para seguir la política óptima.

El entregable principal es el fichero river_mdp.py, que contiene toda la implementación.

5.2. Implementación del entorno (class RiverEnvironment)

El entorno se representa como una cuadrícula de tamaño fijo, donde cada celda puede ser:

- orilla |
- río seguro R
- isla (I, obstáculo)
- salida E

La posición del agente se guarda como una tupla (fila, columna).

Así, para generar el mapa, nos hemos basado en el estilo de generación de mapas que hemos ido realizando a lo largo del proyecto: creando la clase RiverEnvironment (similar a

los fichos map.py y palacio_environment.py). En ella, hemos tanto definido los elementos mencionados, como construido una plantilla base donde colocarlos de manera aleatoria, ayudándonos nuevamente de la función: **_generate_random_map** para ponerlos de manera aleatoria (con ayuda de la librería random), y de la función **plantilla** para poder visualizarlo correctamente.

Además, junto con otras funciones auxiliares como: dentro_tablero, is_island, es_meta, … hemos podido asegurarnos de que el agente se mantenga siempre en la cuadrícula propuesta, las islas se no se sitúen en las orillas, los elementos no se solapen entre sí, etc. De igual manera, se genera un vector strength que alacena la fuerza por corriente por columna, siendo 0 en las orillas.

Por último, con el método aplicar_movimiento_determinista, implementamos la física básica del entorno: si el agente intenta salir del tablero o entrar en una isla, permanece en la misma casilla, no se mueve. Este comportamiento se utiliza posteriormente para construir las transiciones estocásticas del MDP. Obtenemos así una cuadrícula con la siguiente forma:

```
(base) C:\Users\adria\Desktop\2° B iMAT\FIA\PROYECTO FINAL>python river_mdp.py
Política (flechas):
→ → → → → → → → ↓
→ → → → ↓ I → → ↓
↓ I → → → → → → ↓
→ → → → → → → → E
→ → → → → → → → ↑
→ → → → → → → → ↑
→ → → → → → → → ↑

Mapa inicial
|  R  R  R  R  R  R  R  |
|  R  R  R  R  I  R  R  |
|  I  R  R  R  R  R  R  |
CWCK  R  R  R  R  R  R  R  E
|  R  R  R  R  R  R  R  |
|  R  R  R  R  R  R  R  |
|  R  R  R  R  R  R  R  |
```

Ya que hemos realizado igualmente al principio de cada ejecución del problema, una plantilla del sentido de la corriente para saber donde guiarnos.

5.3. Modelado del MDP (class RiverMDP)

Tal y como requiere un Proceso de Decisión de Markov (MDP), hemos definido en esta clase los elementos principales necesarios para modelarlo, los cuales, como bien estipula el enunciado son: los estados, las acciones, las transiciones, las recompensas, y el estado inicial y el objetivo.

Así, con sus métodos correspondientes dentro de la clase, hemos definido cada elemento. Los estados corresponden a todas las posiciciones (i,j) del tablero, excepto las islas. Las acciones, así como las recompensas, las hemos definido en el init de la clase.

Para las transiciones, nos hemos apoyado de un método de la clase para definirlas, puesto que aquí está la clave del MDP. Buscamos así: **P(s'| s,a)**

La probabilidad de transición, como sabemos, depende únicamente del estado actual y de la acción. Por lo que, tras definir las coordenadas de la cuadrícula y las direcciones de la corriente, hemos analizado los distintos casos de acciones:

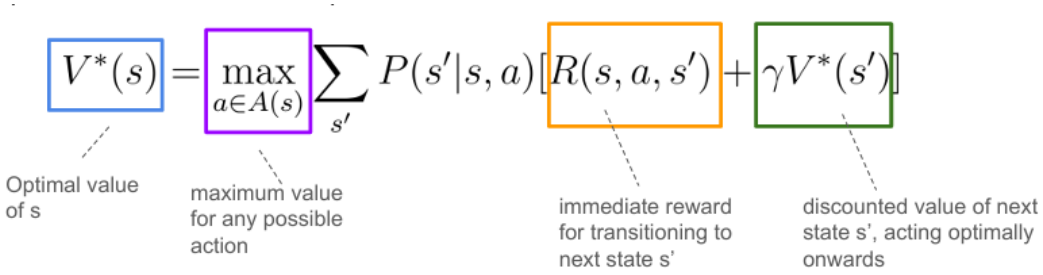
· Si la acción es ‘down’, del agente se mueve hacia abajo con probabilidad = 1.
· Para cualquier otra acción:
-con probabilidad = 1 - river_strength(j), se intenta el movimiento deseado
-con probabilidad = river_strength(j), el agente es empujado hacia abajo por la corriente.
·En el caso de stay, el agente intenta permanecer en la misma posición, pero puede ser empujado igualmente por la corriente.

Las colisiones con bordes o islas, se gestionan automáticamente gracias al rebote del entorno.

Por último, las recompensas, también redefinidas con un método, se alcanzan según estipula el enunciado:
·coste -1: para cada movimiento
·coste -100: por caer en casilla peligrosa
·recompensa +100: por alcanzar la orilla de salida

5.4. Algoritmo de Value Iteration

Se ha implementado el algoritmo de Value Iteration de forma explícita, donde para cada estado, se calcula el valor óptimo aplicando la ecuación de Bellman:



Implementada en la función de nuestro código principalmente con esta línea:

q += p*(mdp.recompensas(s,a,sp) + gamma*V[sp])

Donde para estado s, se evalúan todas las acciones posibles y se selecciona aquella que maximiza el valor esperado, actualizando el valor del estado y almacenamiento posteriormente la acción óptima en la política

Para ello, nos hemos ayudado del pseudocódigo de la implementación del algoritmo:

Algorithm 1: Value Iteration

Result: Find $V^{\pi^*}(s)$ and $\pi^*(s)$, $\forall s$

$V(s) \leftarrow 0,\ \forall s \in S;$
$\Delta \leftarrow \infty;$
while $\Delta \geq \Delta_\circ$ **do**
  $\Delta \leftarrow 0;$
  for *each* $s \in S$ **do**
    $temp \leftarrow V(s);$
    $V(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V(s')];$
    $\Delta \leftarrow \max_a(\Delta, |temp - V(s)|);$
  end
end
$\pi^*(s) \leftarrow argmax_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s'), \forall s \in S;$
return $\pi^*(s), \forall s \in S$

Con esto, en el algoritmo inicialmente la variación máxima de los valores se considera muy grande para asegurar que el proceso iterativo comience. En cada iteración, se recalcula como el mayor cambio en los valores de los estados, y el algoritmo se detiene cuando dicha variación es menor que un umbral pequeño (1e-6 en nuestro código), garantizando así la convergencia.

A partir de los valores finales, se extrae una política greedy, almacenando para cada estado, la acción que maximiza el valor esperado.

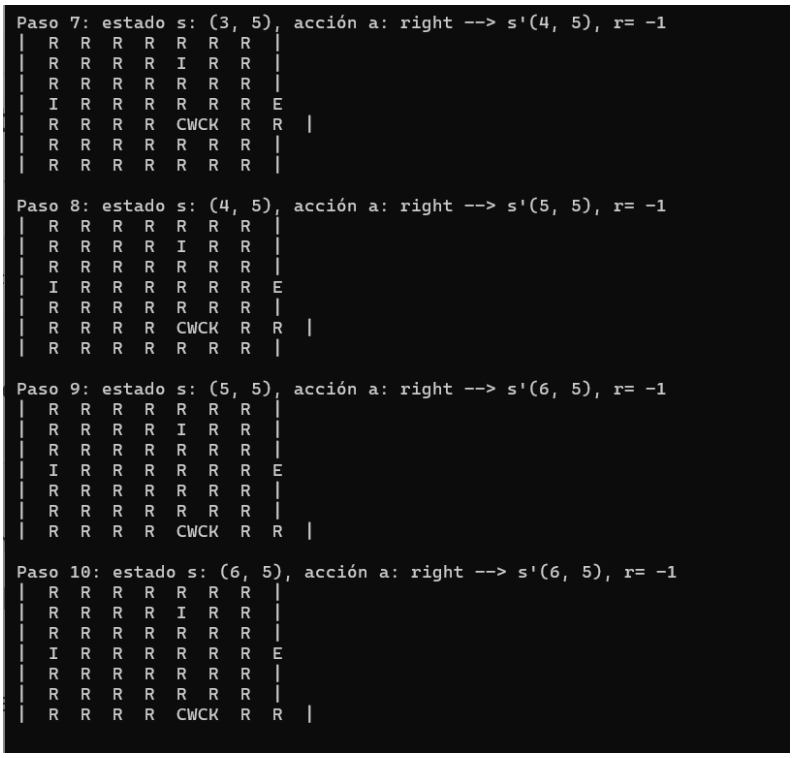
5.5. Simulación siguiendo la política óptima

Una vez obtenida la política, se simulan episodios completos:

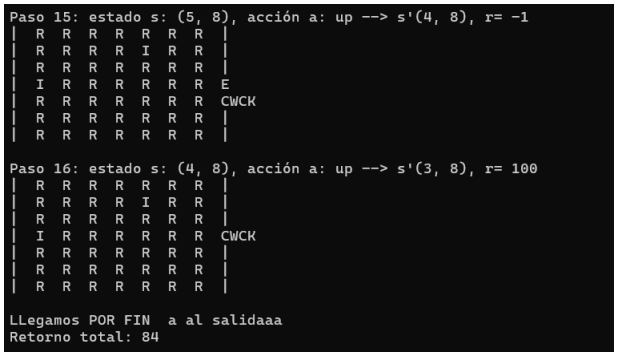
Tras mostrar la plantilla inicial, en cada paso, el agente selecciona la acción indicada por la política. El siguiente estado, se muestrea, con ayuda de una función auxiliar: muestra_siguiente_estado; según la distribución de transición anteriormente mencionada. Por último, se imprime el estado del entorno y el retorno acumulado.

Debido a la estocasticidad de la corriente, distintas ejecuciones pueden producir trayectorias diferentes, aunque la política tiende a conducir al agente hasta la salida en valor esperado.

La implementación muestra así, como un problema con incertidumbre en las acciones, puede resolverse de forma sistemática mediante un MDP y planificación óptima. Un ejemplo de resolución sería:



Tras algún paso más...



Vemos como nos muestra en cada paso, se nos muestra en qué estado estamos, y el agente selecciona la acción indicada por la política óptima obtenida mediante Value Iteration, que maximiza el valor esperado teniendo en cuenta la incertidumbre del entorno.

Si analizamos un ejemplo, como puede ser el paso 7: aunque la acción seleccionada es right, el agente termina en (4,5), es decir, una fila más abajo. Esto ocurre porque, en la columna 5, la fuerza de la corriente (river_strength(5)) es suficientemente alta como para que, con cierta probabilidad el agente sea empujado hacia abajo.

En cada movimiento se aplica una penalización de -1, mientras que al alcanzar la casilla de salida se obtiene una recompensa positiva, finalizando el episodio. El retorno total resulta de así de la suma de estas recompensas/costes, y refleja el equilibrio entre eficiencia del recorrido y éxito en la llegada a la meta.

6. CONCLUSIÓN

A lo largo de este proyecto, he implementado y desarrollado progresivamente un agente capaz de desenvolverse en entornos inciertos y parcialmente observables, aplicando de manera práctica los principales conceptos estudiados en la asignatura.
En la primera parte, se abordó el problema del rescate de Kurtz mediante un razonamiento lógico determinista, construyendo una base de conocimiento KB que permitía inferior información segura a partir de perceptos locales y separar claramente inferencia y toma de decisiones.

En la segunda parte, se amplió este enfoque con un nuevo escenario de trampas y percepciones, además de una nueva resolución basada ahora en inferencia bayesiana, como bien hemos tratado y estudiado profundamente en el bloque 4 de la asignatura (en el que nos hemos inspirado para resolverlo), lo que permitió al agente razonar de forma probabilística sobre la localización de trampas, enemigos, Kurtz y la salida, y asumir riesgos controlados cuando la certeza absoluta no era posible.

Ambas partes las hemos resuelto también con el razonamiento mediante búsqueda no informada (como BFS) e informada (como A*) que estudiamos en el bloque 2 de la asignatura.

Finalmente, en la última parte del proyecto, el problema del cruce del río se modelo como un Proceso de Decisión de Markov, aplicando técnicas de planificación óptima como Value Iteration para obtener políticas óptimas bajo incertidumbre.

En conjunto, el proyecto ha permitido afianzar los conceptos teóricos vistos en clase (lógica, probabilidad, búsqueda y MDPs) desde una perspectiva aplicada y coherente, proporcionando una visión global de cómo se integran distintos paradigmas de la IA en una realidad virtual y en el diseño de agente racionales.

7. BIBLIOGRAFÍA

-Algoritmo Value Iteration:

https://core-robotics.gatech.edu/2021/01/19/bootcamp-summer-2020-week-3-value-iteration-and-q-learning/

-Temario de Fundamentos de Inteligencia Artificial, 2º B de iMAT.

-Resolución de dudas y errores: OpenAI

(cómo qué hacer para hallar el objetivo concreto de A* sin contar con la probabilidad de Kurtz, etc).

ADRIANA LORENZO MINGORANCE, 2ºB de IMAT.